

R CODE -

```
# Load required libraries
```

```
library(tidyverse)
```

```
library(ggplot2)
```

```
library(broom)
```

```
library(caret)
```

```
# Read the dataset
```

```
auto_mpg <- read.csv("C://Users//rable//Downloads//auto-mpg (1).csv", na.strings = "?")
```

```
# Handle missing values (replace '?' with NA and impute)
```

```
auto_mpg <- auto_mpg %>%
```

```
  mutate(
```

```
    horsepower = as.numeric(horsepower),
```

```
    horsepower = ifelse(is.na(horsepower), median(horsepower, na.rm = TRUE), horsepower)
```

```
  )
```

```
# 1. Initial Data Exploration
```

```
summary(auto_mpg)
```

```
str(auto_mpg)
```

```
# Basic visualizations for exploration
```

```
# Scatter plot: MPG vs Horsepower
```

```
ggplot(auto_mpg, aes(x = horsepower, y = mpg)) +
```

```
  geom_point() +
```

```
theme_minimal() +  
labs(title = "MPG vs Horsepower", x = "Horsepower", y = "MPG")
```

```
# Scatter plot: MPG vs Weight  
ggplot(auto_mpg, aes(x = weight, y = mpg)) +  
  geom_point() +  
  theme_minimal() +  
  labs(title = "MPG vs Weight", x = "Weight", y = "MPG")
```

```
# Boxplot: MPG by Cylinders  
ggplot(auto_mpg, aes(x = as.factor(cylinder), y = mpg)) +  
  geom_boxplot() +  
  theme_minimal() +  
  labs(title = "MPG by Number of Cylinders", x = "Cylinders", y = "MPG")
```

```
# 2. Split dataset: First 300 rows for training, remaining for testing
```

```
train_data <- auto_mpg[1:300, ]  
test_data <- auto_mpg[301:nrow(auto_mpg), ]
```

```
# 3. Simple Linear Regression (MPG vs Horsepower)
```

```
simple_lm <- lm(mpg ~ horsepower, data = train_data)  
summary_simple_lm <- summary(simple_lm)
```

```
# Extract metrics
```

```
simple_r_squared <- summary_simple_lm$r.squared  
simple_adj_r_squared <- summary_simple_lm$adj.r.squared # <-- FIXED
```

```
simple_equation <- paste0(
  "mpg = ", round(coef(simple_lm)[1], 2),
  " + ", round(coef(simple_lm)[2], 2), " * horsepower"
)
```

4. Multiple Linear Regression (MPG vs multiple features)

```
multiple_lm <- lm(mpg ~ cylinder + displacement + horsepower + weight +
  acceleration + model.year + origin, data = train_data)
```

```
if (!inherits(multiple_lm, "lm")) {
  stop("Error: Multiple linear model failed to create.")
} else {
  cat("Multiple linear model created successfully.\n")
}
```

```
summary_multiple_lm <- summary(multiple_lm)
```

Extract metrics

```
multiple_r_squared <- summary_multiple_lm$r.squared
multiple_adj_r_squared <- summary_multiple_lm$adj.r.squared # <-- FIXED
multiple_coefficients <- coef(multiple_lm)
```

Print metrics

```
cat("Multiple R-squared: ", multiple_r_squared, "\n")
cat("Multiple Adj R-squared:", multiple_adj_r_squared, "\n\n")
```

```
multiple_equation <- paste0(
  "mpg = ", round(multiple_coefficients[1], 2),
  " + ", round(multiple_coefficients[2], 2), " * cylinder",
  " + ", round(multiple_coefficients[3], 2), " * displacement",
  " + ", round(multiple_coefficients[4], 2), " * horsepower",
  " + ", round(multiple_coefficients[5], 2), " * weight",
  " + ", round(multiple_coefficients[6], 2), " * acceleration",
  " + ", round(multiple_coefficients[7], 2), " * model.year",
  " + ", round(multiple_coefficients[8], 2), " * origin"
)
```

5. Log regression metrics into a data frame

```
regression_log <- data.frame(
  Model = c("Simple Linear", "Multiple Linear"),
  R_Squared = c(simple_r_squared, multiple_r_squared),
  Adjusted_R_Squared = c(simple_adj_r_squared, multiple_adj_r_squared),
  Equation = c(simple_equation, multiple_equation),
  stringsAsFactors = FALSE
)
```

```
print(regression_log)
```

6. Prediction on Test Data (using multiple linear model)

```
test_data$predicted_mpg <- predict(multiple_lm, newdata = test_data)
test_data$residuals <- test_data$mpg - test_data$predicted_mpg
```

7. Model Validation Metrics

```
rmse <- sqrt(mean(test_data$residuals^2))
```

```
mae <- mean(abs(test_data$residuals))
```

```
cat("Test Data RMSE:", rmse, "\n")
```

```
cat("Test Data MAE: ", mae, "\n\n")
```

8. Residual Plot

```
ggplot(test_data, aes(x = predicted_mpg, y = residuals)) +  
  geom_point() +  
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +  
  theme_minimal() +  
  labs(title = "Residual Plot", x = "Predicted MPG", y = "Residuals")
```

9. Histogram of Residuals

```
ggplot(test_data, aes(x = residuals)) +  
  geom_histogram(bins = 20, fill = "blue", alpha = 0.5) +  
  theme_minimal() +  
  labs(title = "Histogram of Residuals", x = "Residuals", y = "Frequency")
```

10. Save results for report

```
write.csv(regression_log, "regression_metrics.csv", row.names = FALSE)  
write.csv(test_data[, c("mpg", "predicted_mpg", "residuals")],  
          "predictions.csv", row.names = FALSE)
```

11. Export visualizations for PowerPoint/Tableau

```
ggsave("mpg_vs_horsepower.png",  
  plot = ggplot(auto_mpg, aes(x = horsepower, y = mpg)) +  
    geom_point() + theme_minimal() +  
    labs(title = "MPG vs Horsepower"))
```

```
ggsave("mpg_vs_weight.png",  
  plot = ggplot(auto_mpg, aes(x = weight, y = mpg)) +  
    geom_point() + theme_minimal() +  
    labs(title = "MPG vs Weight"))
```

```
ggsave("residual_plot.png",  
  plot = ggplot(test_data, aes(x = predicted_mpg, y = residuals)) +  
    geom_point() + geom_hline(yintercept = 0, linetype = "dashed") +  
    theme_minimal() + labs(title = "Residual Plot"))
```

```
ggsave("residual_histogram.png",  
  plot = ggplot(test_data, aes(x = residuals)) +  
    geom_histogram(bins = 20, fill = "blue", alpha = 0.5) +  
    theme_minimal() + labs(title = "Histogram of Residuals"))
```

12. Optional: Export full dataset for Tableau

```
write.csv(auto_mpg, "auto_mpg_for_tableau.csv", row.names = FALSE)
```

13. Print key results for report

```
cat("=== Simple Linear Regression ===\n")
```

```
cat("R-squared:      ", simple_r_squared, "\n")
```

```
cat("Adjusted R-squared: ", simple_adj_r_squared, "\n")
```

```
cat("Equation:      ", simple_equation, "\n\n")
```

```
cat("=== Multiple Linear Regression ===\n")
```

```
cat("R-squared:      ", multiple_r_squared, "\n")
```

```
cat("Adjusted R-squared: ", multiple_adj_r_squared, "\n")
```

```
cat("Equation:      ", multiple_equation, "\n\n")
```

```
cat("=== Test Data Metrics ===\n")
```

```
cat("RMSE:          ", rmse, "\n")
```

```
cat("MAE:           ", mae, "\n")
```